

VentureMiner AI

Plugin Architecture

Document: 07-AI-Architecture / 13_plugin_architecture

Version: v1.0

Date: 2026-07-20

Author: VentureMiner AI Documentation Team

Status: Approved

Project: VentureMiner AI — AI Venture Intelligence Platform

Table of Contents

Document 13 — Plugin Architecture

Table of Contents

1. Purpose & Scope
2. Why plugins
3. Plugin anatomy
4. Manifest
5. Lifecycle
6. Sandboxing
7. Registry
8. Distribution
9. Security model
10. Versioning & compatibility
11. Observability
12. Failure modes
13. Appendix
 - 13.1 Glossary
 - 13.2 Revision history
 - 13.3 Cross-references

Document 13 — Plugin Architecture

A plugin is a packaged tool that extends the platform's reach. This document defines the lifecycle, the sandbox, the registry, and the security model.

Table of Contents

1. Purpose & Scope
2. Why plugins
3. Plugin anatomy
4. Manifest
5. Lifecycle
6. Sandboxing
7. Registry
8. Distribution
9. Security model
10. Versioning & compatibility
11. Observability
12. Failure modes
13. Appendix

1. Purpose & Scope

This document is the contract for plugins — first-class, versioned, sandboxed tools that agents can call via the MCP gateway. It defines what a plugin is, how it ships, how it runs, and how it fails.

2. Why plugins

- Extensibility. A new source (e.g. a niche industry database) ships as a plugin, not a service.
- Customer-specific logic. Enterprise customers can ship private plugins without forking the platform.
- Safe third-party. A registry with signed, reviewed plugins gives the marketplace a place to grow.

3. Plugin anatomy

A plugin is a directory:

```
my-plugin/
├── plugin.yaml # manifest
├── README.md
├── src/
│   ├── handler.py # tool logic
│   └── tests/
└── signatures/ # detached signatures
```

4. Manifest

```
id: T-NICHE-INDUSTRY-DB
name: Niche Industry DB
version: 1.0.0
vendor: example.com
risk_level: low
pii_risk: false
entrypoint: src/handler.py:handle
tools:
- name: niche_search
description: Search the industry database.
input_schema: { ... }
output_schema: { ... }
permissions:
- network:outbound=api.example.com
- filesystem:read=./cache
secrets:
- secret_ref: provider/example/api_key
homepage: https://example.com
license: Apache-2.0
```

5. Lifecycle

Draft → Submitted → Review → Approved → Published → Active → (Deprecated | Retired)

- Draft — author writes manifest + handler.
- Submitted — sent to the registry.
- Review — automated (signature, schema) + manual (security review for high-risk).
- Approved — passed review; awaiting publish.
- Published — listed in the public registry.
- Active — installed in one or more workspaces; callable.
- Deprecated — no new installations; existing continue to work.
- Retired — removed; existing installations receive a manifest-version-mismatch error.

6. Sandboxing

- Plugins run in an isolated process (container, VM, or wasm runtime).
- Default: per-plugin container with no network by default; outbound explicitly allow-listed in manifest.
- Filesystem: read-only except for declared scratch space.
- Memory: capped.

- High-risk plugins (e.g. arbitrary network access) require elevated review.

7. Registry

- Public registry: curated list of approved plugins.
- Private registries: per-workspace, for Enterprise custom plugins.
- Storage: S3-compatible object store, indexed in Postgres.
- Signatures: detached signatures (Sigstore-style); verified on install.

8. Distribution

- Public plugins: downloadable from the public registry.
- Private plugins: uploaded to a workspace registry; signed by the workspace owner.
- Bundled plugins: shipped with the platform (e.g. T-MARKET-DATA-FETCHER for paid data providers).

9. Security model

- Trust is scoped. A plugin is only as trusted as its signature and its review.
- Per-workspace enablement. Workspace admins decide which plugins their members can call.
- Per-run allow-list. A run only sees the tools its orchestrator grants.
- Audit log. Every plugin call is logged.
- Kill switch. Admin can disable a plugin globally; existing calls are aborted.

10. Versioning & compatibility

- Plugins use semver.
- Major bump = breaking input/output schema or permissions.
- Minor bump = additive, backwards-compatible.
- Patch = bug fix.
- Agents declare a manifest-version range; the gateway enforces it.

11. Observability

- Per-plugin metrics: call rate, error rate, p95 latency, cost.
- Per-plugin traces: every call is a span.
- Per-plugin logs: structured, PII-redacted.
- Dashboards surfaced in the admin panel.

12. Failure modes

Plugin crashes	Container restart; retries; if persistent, quarantine.
Plugin schema drift	Manifest mismatch error; alert; suspend plugin.
Plugin security issue	Kill switch; public advisory; force-uninstall.
Plugin over budget	Rate-limit; warn admin.
Plugin signature invalid	Reject install.
Plugin out of date	Warn; allow time-bound grace.

13. Appendix

13.1 Glossary

Plugin	A packaged tool with a manifest, handler, and signature
Manifest	The declarative description of a plugin
Registry	A repository of approved plugins
Risk level	low / medium / high — gates review and sandboxing

13.2 Revision history

v0.5	2026-07-20	Doc Team	All sections drafted
v1.0	2026-07-20	Doc Team	First approved version

13.3 Cross-references

- MCP: Document 12.
- Multi-Agent: Document 08.
- Security: Document 21.

End of Document 13 — Plugin Architecture. The plugin is the unit of extensibility; everything outside the platform is a plugin.